Department of Computer Science

CHAPTER ONE: INTRODUCTION TO

PRESENTED BY:
BIRUK M.

1.1 Software

- Software systems are complex creations, they are hard to understand, and they are never finished-Vapor ware!
- they comprise many components; many of their components are custom made and complex themselves;
- **Software types:**
  - Custom
    - **For a specific customer**
  - Generic
    - **Sold on open market**
    - **Often called**
      - COTS (Commercial Off The Shelf)
      - Shrink-wrapped
  - Embedded
    - **Built into hardware**
    - **Hard to change**

The Nature of Software:

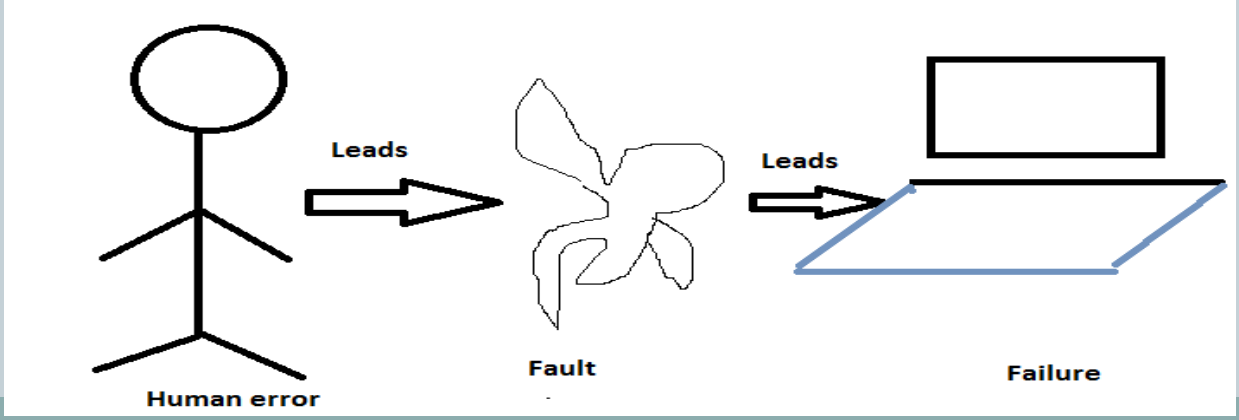
- **Software is intangible**
  - Hard to understand development effort
- **Software is easy to reproduce**
  - Cost is in its *development*
    - in other engineering products, manufacturing is the costly stage
- **The industry is labor-intensive**
  - Hard to automate
- **Untrained people can hack something together**
  - Quality problems are hard to notice
- **Software is easy to modify**
  - People make changes without fully understanding it
- **Software does not 'wear out'**
  - It *deteriorates* by having its design changed:
    - erroneously, or
  - in ways that were not anticipated, thus making it complex

Cont…

- Conclusion:
  - Many software has poor design and is getting worse
  - Demand for software is high and rising
  - We are in a perpetual 'software crisis'
  - We have to learn to 'engineer' software

- **Problems:** sometimes problem has nothing to do with the computer, human error (programmer, designer, user error) may lead to faulty program and then to failure.

Pic1.

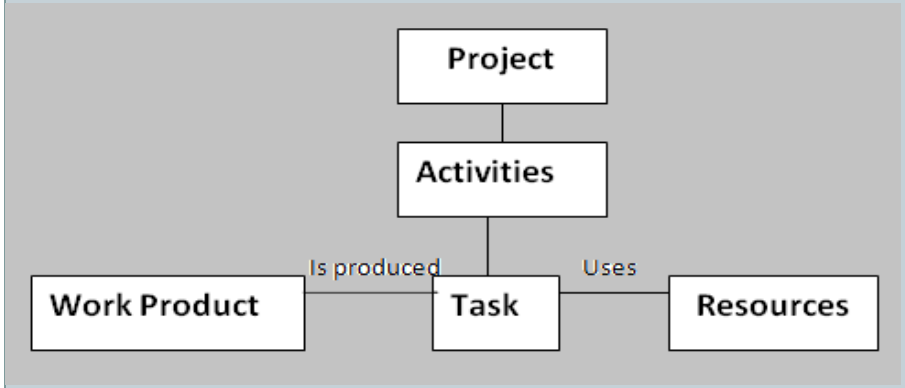


1.2 Project and software project mgmt

- **Software process**: is a set of activities that is performed towards a specific purpose (e.g, requirements, management, or delivery).
  - A software process is a road map that helps you create a timely, high quality result.
  - All activities and associated results which produce software Usually involves a set of tools and techniques.
- **Project:** an organized undertaking which consists of number of activities.
  - Helps to materialize the software development process.
- **Activity:** composed of a number of Tasks.

Cont…

- **Tasks:** works in terms of sequential steps. Each task has a work product.
- **Work Product:** an intermediate product or deliverables of the project
  - It uses resources and could produce system model/document



What Is a Project?

A temporary endeavor undertaken to create a unique product, service, or result.

- has a definite beginning and end

The end is reached

- when the project's objectives have been achieved
- terminated because its objectives will not or cannot be met
- terminated at the customer's (sponsor's) choice

Project mgmt.

- Project management is a distinct area of management that helps in handling projects.
- It has three key features to distinguish it from other forms of management
  - project manager,
  - the project team and
  - the project management system.
- The project management system comprises organization structure, information processing and decision making and the procedures that facilitate integration of horizontal and vertical elements of the project organization. focuses on integrated planning and control
- Project management is the application of methodology, tools, and processes to effectively design and execute projects. Project management utilizes groups and assets to finish project exercises within the limits of time, cost, and extension
- Project management process is **an administration process for the planning and control of the services or the implementation of a project**

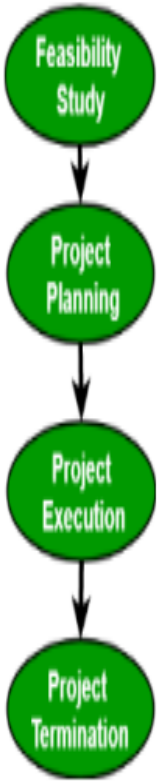
Project mgmt

- The results of one of these processes are:

1. delivery of the project product
2. achievement of the project objectives
3. documentation of the learning processes **conceptualization phase=feasibility study

- Project mgmt. process

- Feasibility study
- Project Planning
- Project Execution
- Project Termination



1.3 Software life cycle models

- The process of solving customers' problems by the systematic development and evolution of large, high-quality software systems within cost, time and other constraints is called Software Engineering
- It is a systematic approach to software development
- It has to model both the problem and the solution domain (to understand the environment and to evaluate different solutions that the system could build)
- **Why modeling?** Because
  - Models are an abstract representation of a system that enables us to answer questions about the system.
  - Helps to visualize and understand the system
  - Used to search for an acceptable solution

Cont…

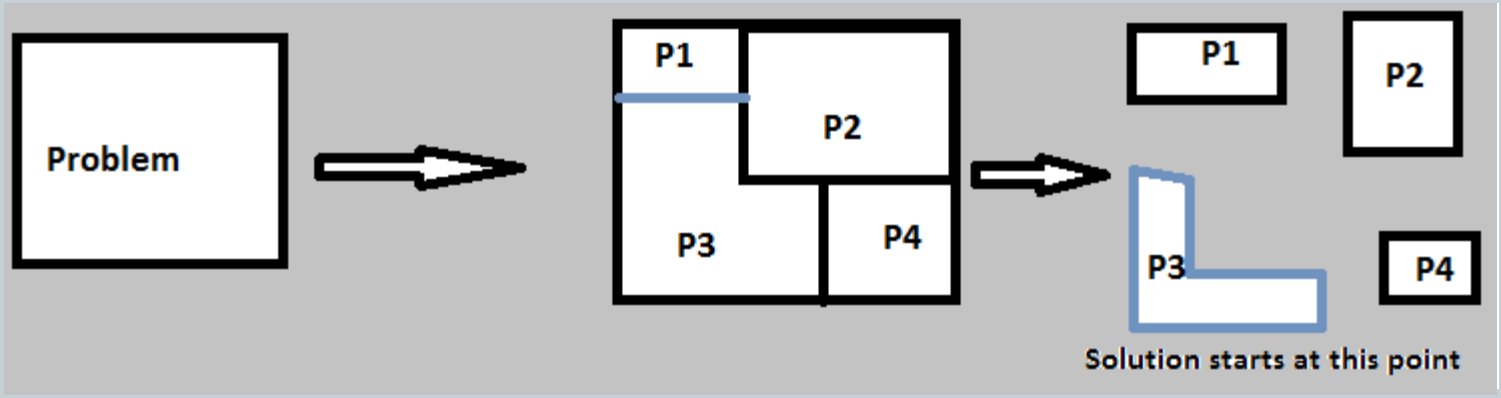
- Models of the software development process are called **software life cycle models.**
- A software life cycle model represents all the activities and work products necessary to develop a software system.
- Life cycle models enable managers and developers to deal with the complexity of the process of developing software in the same way as an analysis model or a system design model enables developers to deal with the complexity of a software system.
- In the case of software systems, the reality being modeled includes phenomena such as watches, accidents, trains, sensors, and buildings. In the case of software development, the reality includes phenomena such as participants, teams, activities, and work products.

- Generally, problem solving techniques must have two parts

1. Analysis

- Breaking the problem into pieces
- Used to understand and deal with the problem
- Describing the large problem as a collection of small problems

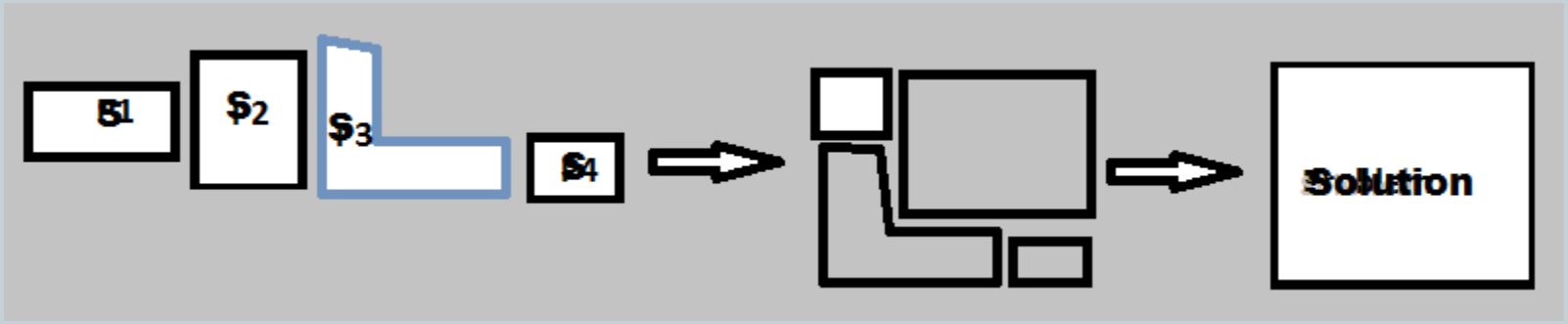
Pic2.



2. Synthesis

- The reverse process of analysis
- Putting together small blocks into a large structure
- It is a composition of individual solutions
- It is a challenging task as a finding solution

Pic3



- Note: -In solving problems we employ a variety of methods, tools, processes, paradigms, procedures etc
- -Software engineering teaches us the tools, techniques, procedures and paradigms to enhance the quality of the software product.

phases

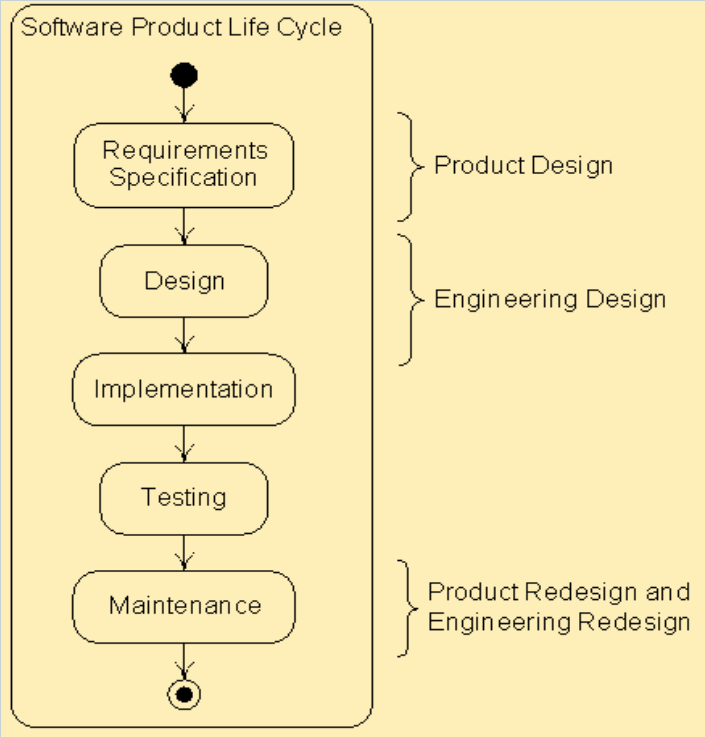
- *The five phases of the project life cycle are initiation, planning, execution, monitoring and controlling, and closing. They represent the sequential steps from project conception to completion, each essential for successful [project management](#).*

Five phases of project management life cycle

1. **Project Initiation**
2. **Project Planning**
3. **Project Execution**
4. **Project Monitoring and Control**
5. **Project Closure**

Phases-stages

- Life cycle models make the software development activities and their dependencies visible and manageable.
  - Life cycle model breaks down the development process into phases or stages.
  - Each phase can be performed in various ways.
- In software development the followings are the major software process activities
  - Requirement elicitation and analysis
  - System design
  - Object design'
  - Program Implementation (coding)
  - Unit and Integration Testing
  - System Testing
  - System Delivery
  - Operation and Maintenance
- Note: each stage by itself is a process.



phases

Project Life Cycle

A series of phases that a project passes through from its initiation to its closure.

Phases are generally time bounded, with a starting and ending or control point.

Range from predictive (plan-driven) to adaptive (change-driven) approaches.

- **Predictive: product and deliverables are defined at the beginning of the project and any changes to scope are carefully managed.**
- **Adaptive: product is developed over multiple iterations and detailed scope is defined for each iteration only as the iteration begins.**

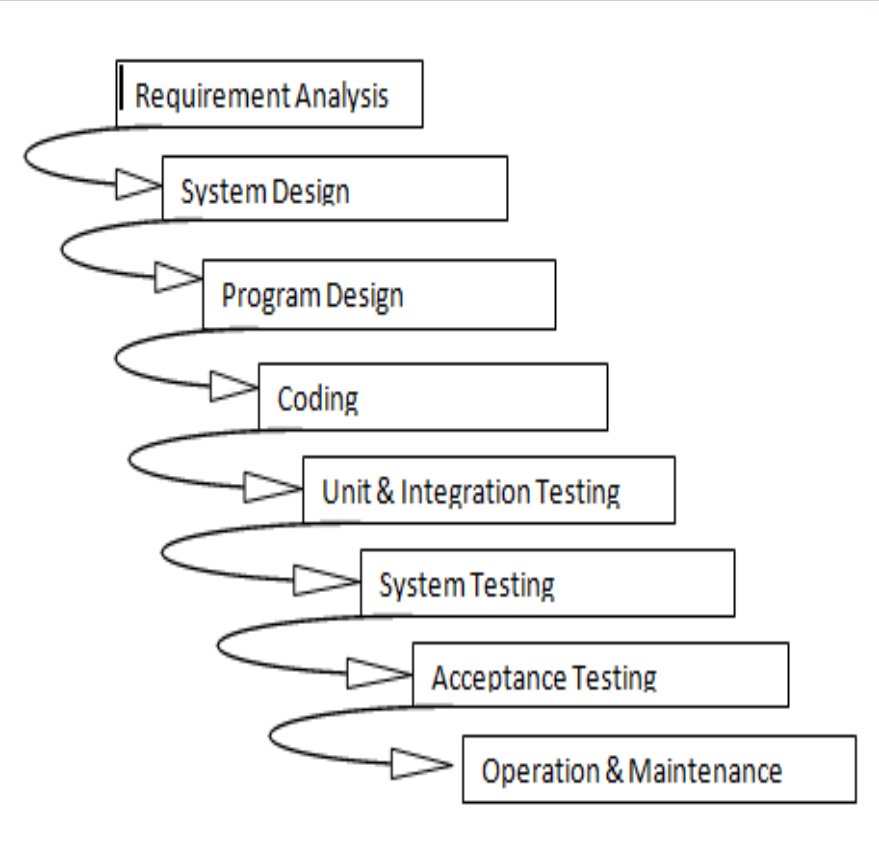
- There are different types of software process models such as:-

1. Waterfall /sequential model

- The model assumes that software development can be scheduled as a step-by-step process that transforms user needs into code
  - The stages are depicted as cascading from one to another.
  - One development stage should be completed before the next begins.
  - It is very useful to present a very high level view of the development.
  - It is easy and simple to use the model
  - Used to explain the customers who are not familiar with software development
  - an activity-centered life cycle model that prescribes a sequential execution of a subset of the development processes

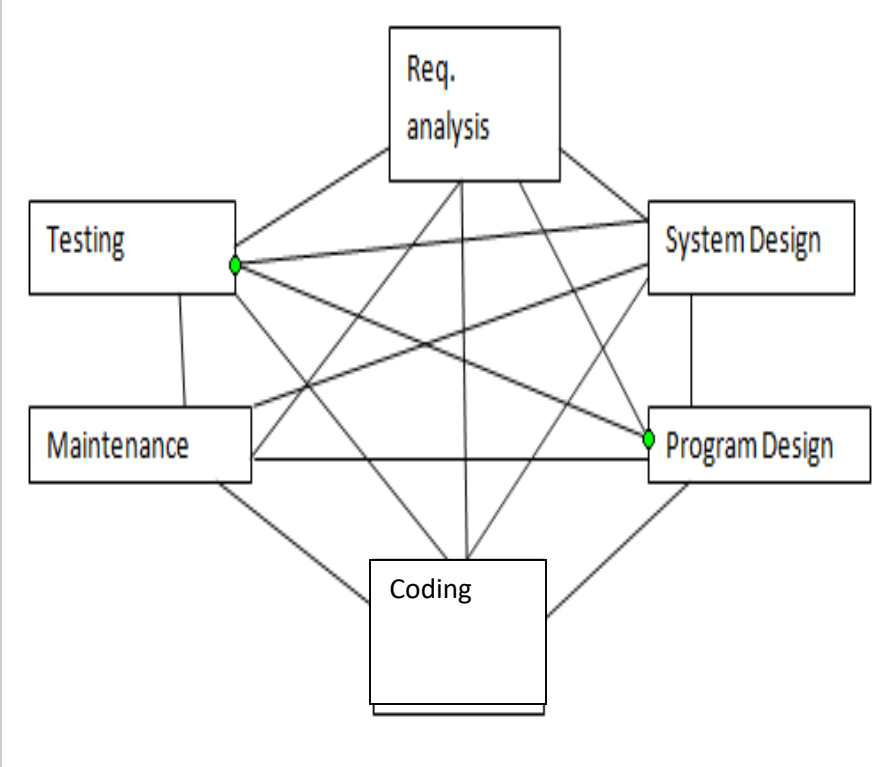
- Drawbacks
  - It does not show the way code is really developed
  - Actual software development process will not always sequential
  - The model provides no guidance to developers, managers,…on how to handle changes to activities
  - The customer must have patience---late delivery
  - The model tells nothing about the typical back and forth activities

- Note:
  - it will be a reasonable approach for the projects in which the requirements are well understood.



Pic2:

Software development in reality, all are connected



2. V-model

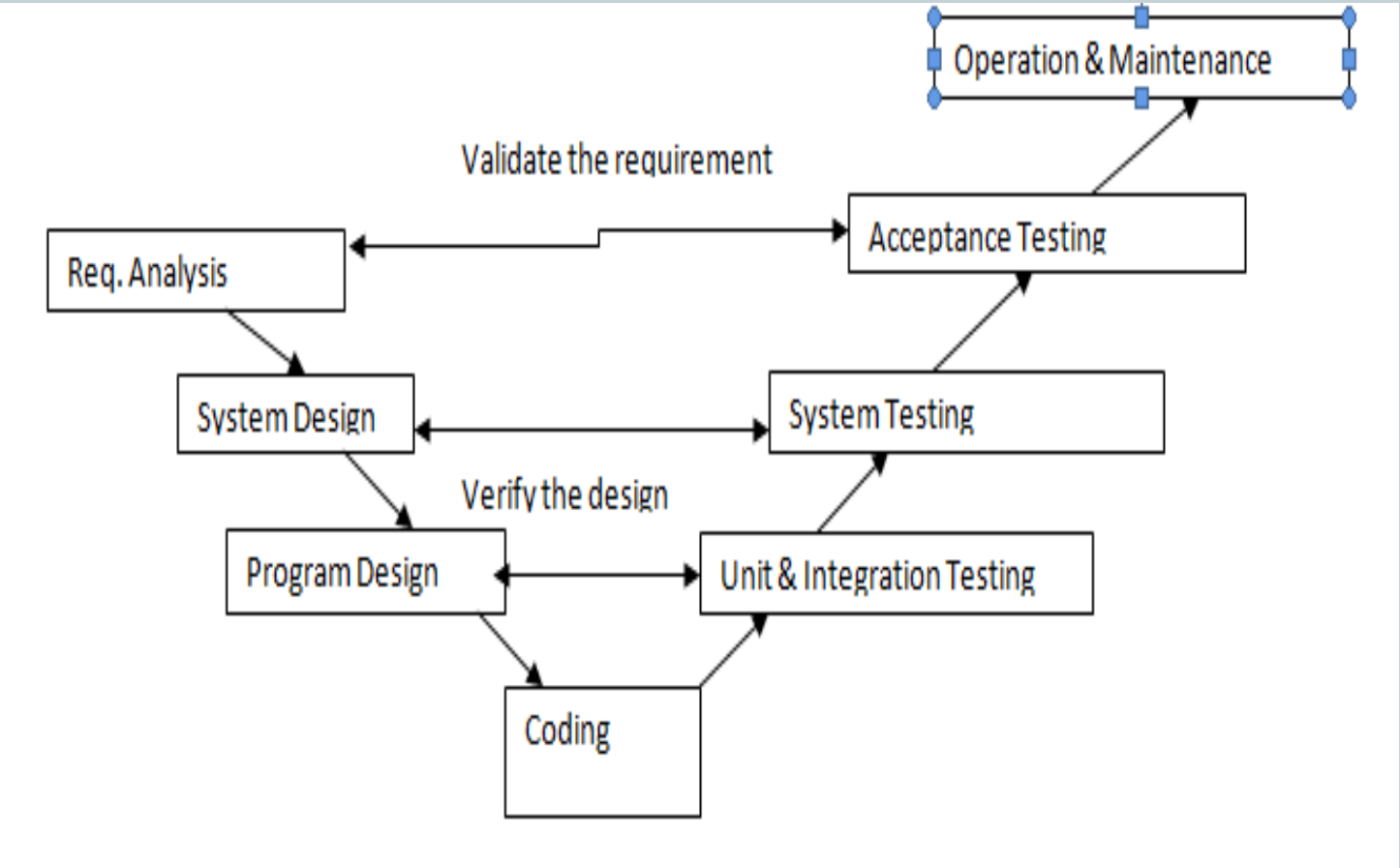
- The **V-model** is a variation of the waterfall model that makes explicit the dependency between development activities and verification activities.

- The difference between the waterfall model and the V model is that
  - V-model makes explicit the notion of level of abstraction.
  - All activities from requirements to implementation focus on building more and more detailed representation of the system, whereas all activities from implementation to operation focus on validating the system
  - It shows the relation of the design with test activities.

- The analysis and design on one side and testing maintenance on the other side.
- Unit and Integration testing should verify the program design.
- System testing also should verify the system design
- Acceptance test conducted by customer should validate the requirement.

- **Note:**
  - If problems are found in verification and validation, the left side can be re-executed to fix and improve the design.
  - The V-model makes more explicit some of the interactions and rework that are hidden in the waterfall model.

- Pic4-V-MODEL


Operation & Maintenance
Validate the requirement
Req. Analysis
Acceptance Testing
System Design
System Testing
Verify the design
Program Design
Unit & Integration Testing
Coding

3. Prototyping model (Rapid Prototyping model)

- Prototype is a sub process. (**First system-** a mechanism for identifying SW requirements)
- It is the act of building a **small-scale**, representative or working model of the users' requirements.
- It is a technique for building a functional model of the information system using rapid application development tools.
- A rapid prototype is a **working model** that is functionally equivalent to a subset of the product (internal structure is not concerned yet).

Cont…

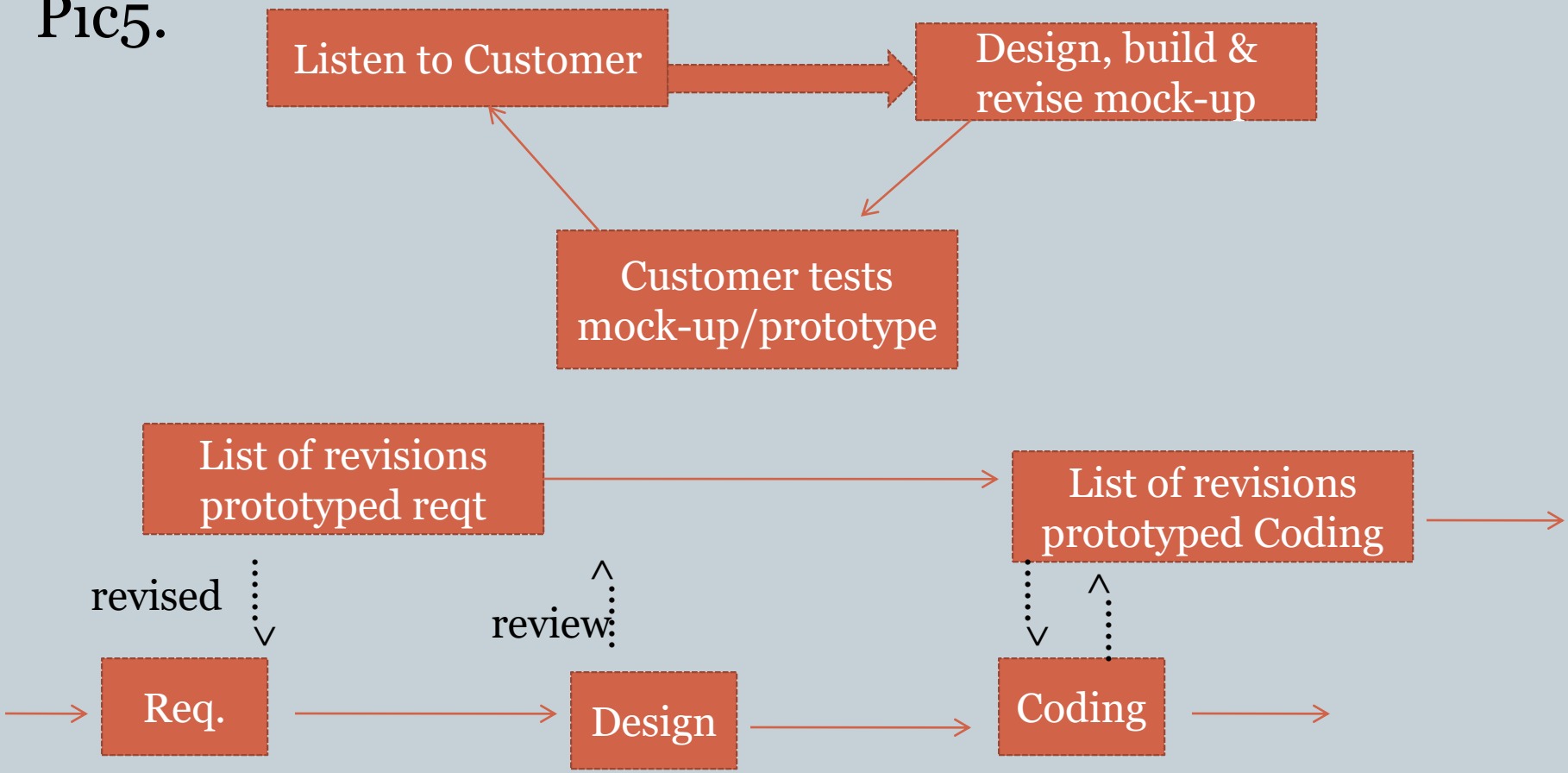
- The sole use of rapid prototyping is to determine what the **client's real needs are**, construct the rapid prototype as rapidly as possible to speed up the s/w development process.
- It considers a partial process, examine some aspects, and decide if it is suitable for the finished product.
- Used to check if it is consistent, feasible, practical, if not revisions are made at early stage rather than at the end.
- Helps the developers asses' alternative design strategies and decide what is best for the project

Cont…

- It is considered to be an efficient model and used with the other models to improve understanding.
- Encourages and requires active **users participation**
- Prototyping can increase **creativit**y because it allows for quick user feedback, which can lead to better solutions.
- Can consolidate parts of phases that normally occur one after the other
- When the customer has a legitimate need but is clueless about the details, develop a prototype as first step.
- Evolves through iteration into the required system.

Cont…

Pic5.


Listen to Customer
Design, build & revise mock-up
Customer tests mock-up/prototype
List of revisions prototyped reqt
List of revisions prototyped Coding
revised
review
Req.
Design
Coding

4. Evolutionary Model

- Business and product requirements often change as development proceeds, making a straight path to an end product unrealistic. In such situations, software engineers need a process model that has been designed to accommodate a product that evolves over time.

- Evolutionary models are interactive.

- They are characterized in a manner that enables software engineers to develop increasingly more complete version of the software.
- Examples of the evolutionary model are:

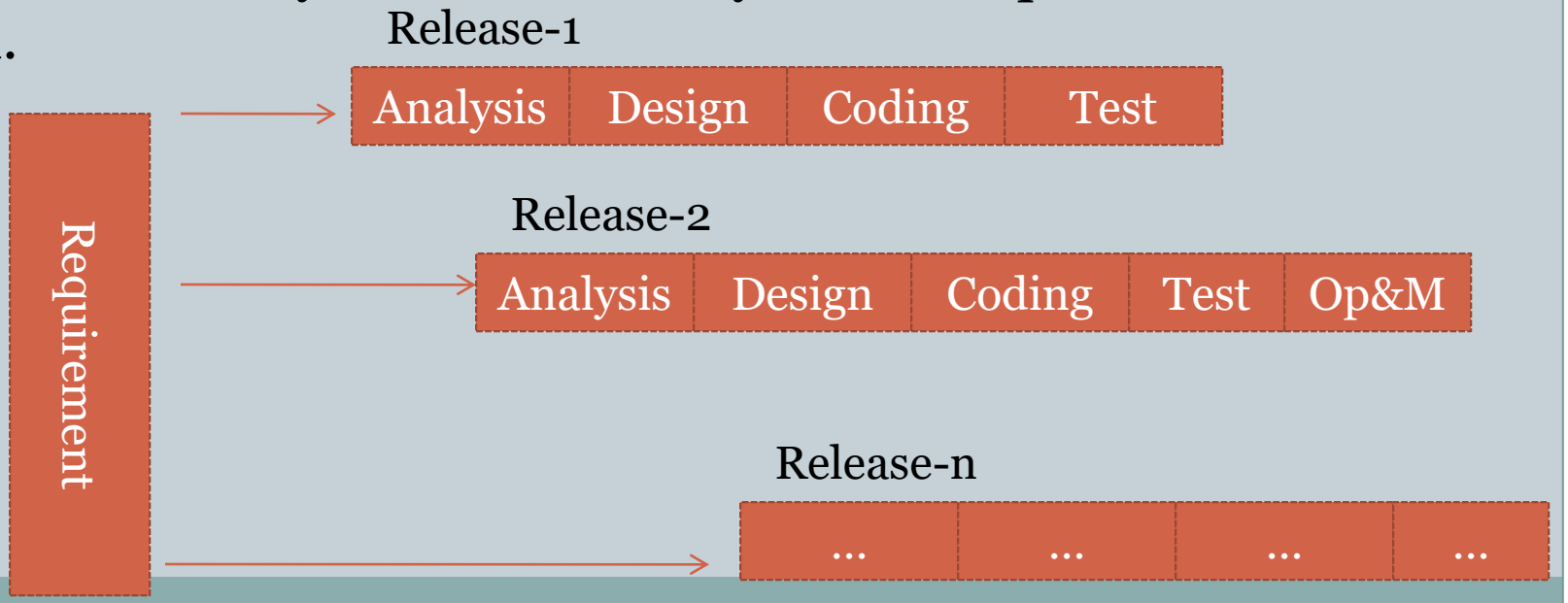
a. Incremental model

- Partitioned into subsystem by functionality
- From release 1 to release…n (each release adds more functionality)
- The first product is often a core product in which the basic requirements are addressed.
- When you encounter a **difficult deadline** that cannot be changed, the incremental model is a good paradigm to consider.
- The incremental model delivers software in small but usable pieces, called increments.
- Each increment builds on that have already been delivered.
- The model combines elements of **the linear sequential model** with **iterative philosophy of prototyping**.

Cont…

- The s/w product is designed, implemented, integrated, and tested as a series of incremental *builds*, where a build consists of code pieces from various modules interacting to provide a specific functional capability.
- It is sometimes necessary to re-specify, re-design, re-code, or at worst, throw away what has already been completed and start again.

Pic-6



b. Spiral Model

- It couples the **iterative nature of prototyping** with the controlled and **systematic aspects of the linear sequential mode**l.
- Could combine development activities with risk management to minimize and control risk.
  - **The idea of minimizing risk via the use of prototypes and other means is the concept underlying the *spiral model*.**
- A simplified spiral model is as a waterfall model with each phase preceded by risk analysis.
  - Before commencing each phase, an attempt is made to control (resolve) the risks. If it is impossible to resolve all the significant risks at a stage, then the project is immediately terminated.
  - the software is developed in series of incremental releases.during early iterations, the incremental release might be a paper model or prototype.During latter iterations, increasingly more complete versions of the engineered system are produced.

Cont…

- A spiral model is divided into a number of framework activities called task regions.

1. Customer Communication:-

- tasks required establishing effective communication between developers and customers.

2. Planning:-

- tasks required defining resources, timeliness and other project related information.

3. Risk Analysis:-to asses both technical and managerial risks

4. Engineering:-

- to build one or more representations of the application (analysis & Design)

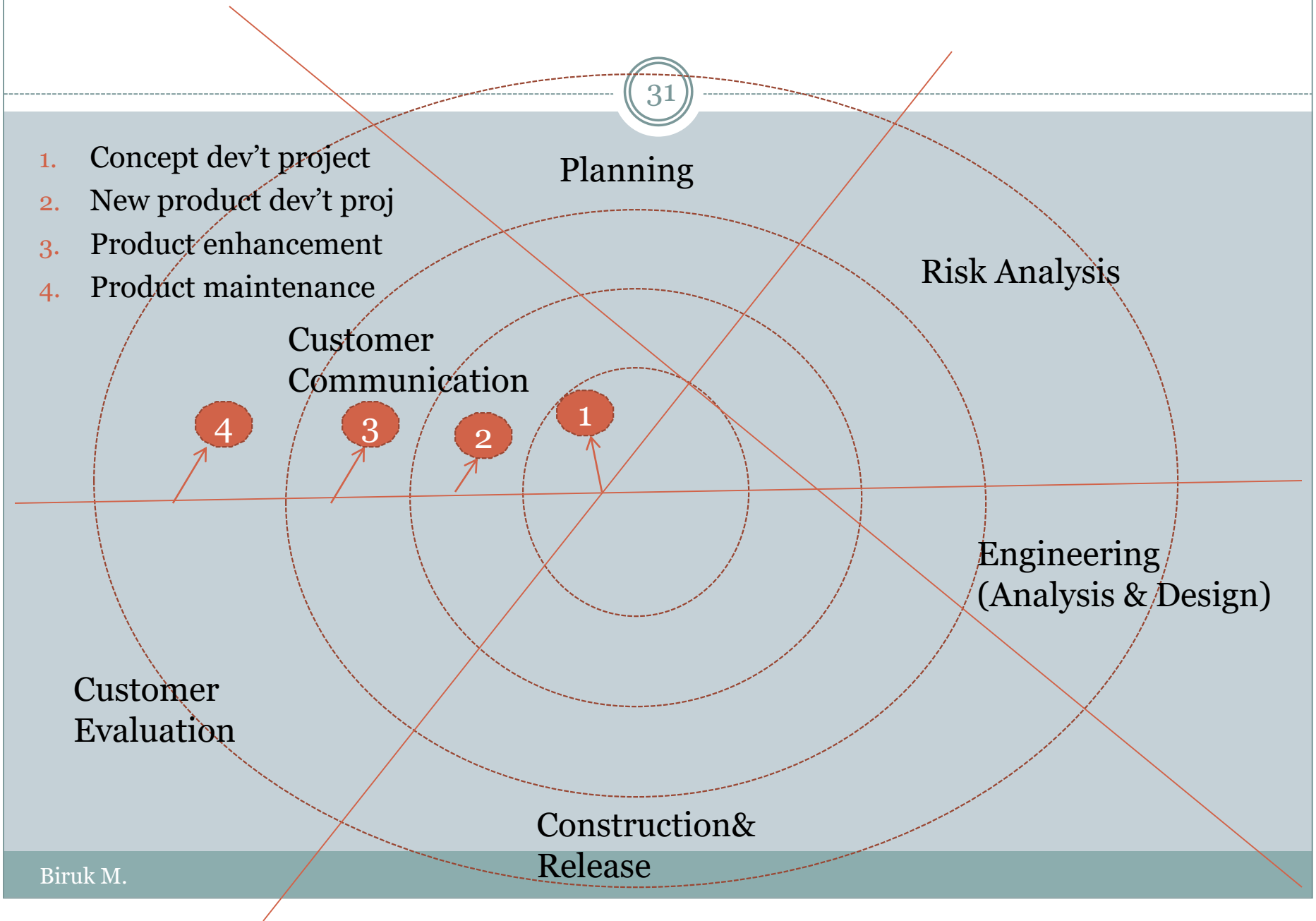
5. Construction and Release:-

- to construct, test, install and provide users support (Documentation and Training)

6. Customer Evaluation:-

- to obtain customer feedback based on evaluation of the software representations created during the engineering stage and implemented during the installation stage.

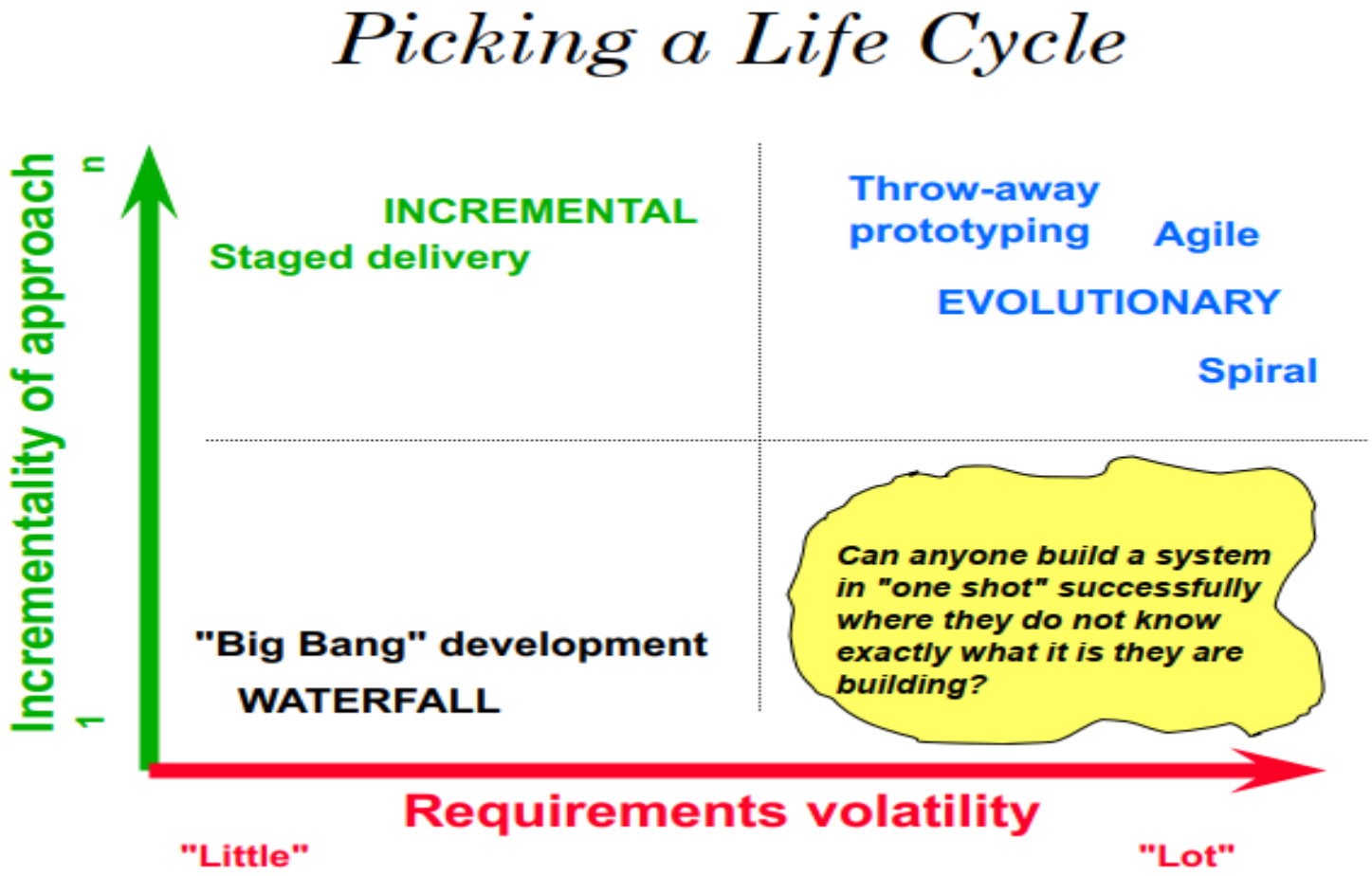
1. Concept dev't project
2. New product dev't proj
3. Product enhancement
4. Product maintenance



Cont…

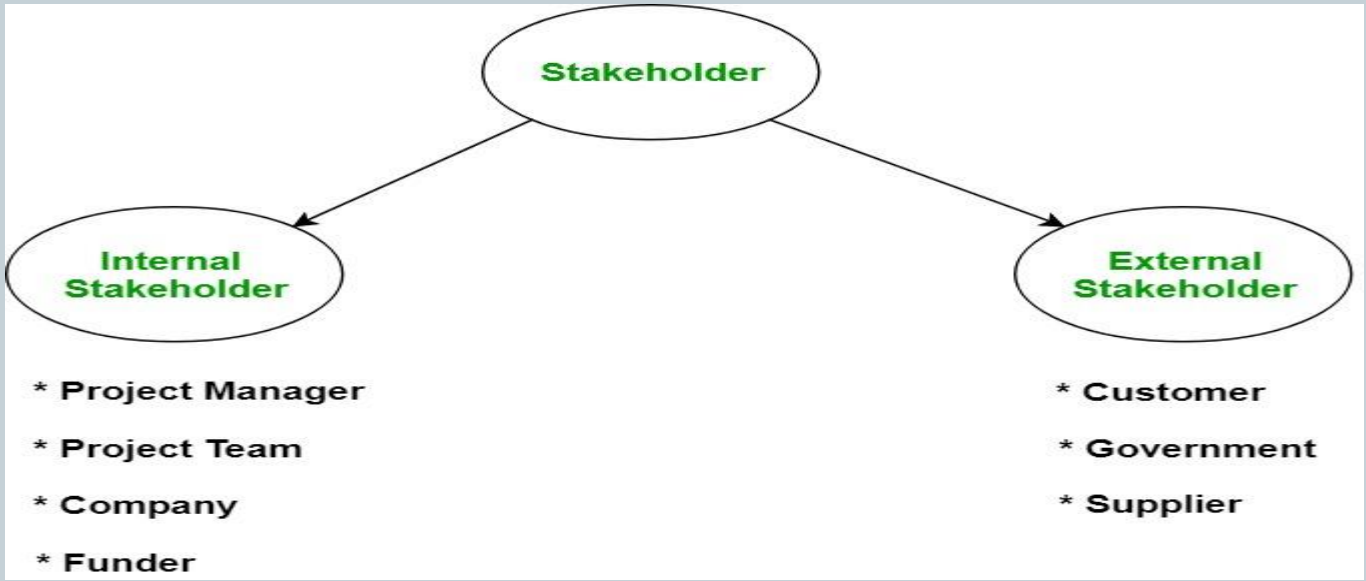
- **Other models**
  - Chaos:-haphazard approach to software development. It is an ad hoc disorganized process model. Depends on trial and error method
  - Built and fix model:-Without specification or any attempt in design, just build a product, and reworked as many times needed to satisfy the customer.Unsatisfactory for any size of s/w development, we better specify the various phases of software process.
- **Conclusion:-**
  - Software is developed using a life cycle model.
  - Just a life cycle model is insufficient for development.

conclusion


Picking a Life Cycle
Incrementality of approach
n
1
INCREMENTAL
Staged delivery
Throw-away prototyping
Agile
EVOLUTIONARY
Spiral
"Big Bang" development
WATERFALL
Can anyone build a system in "one shot" successfully where they do not know exactly what it is they are building?
Requirements volatility
"Little"
"Lot"

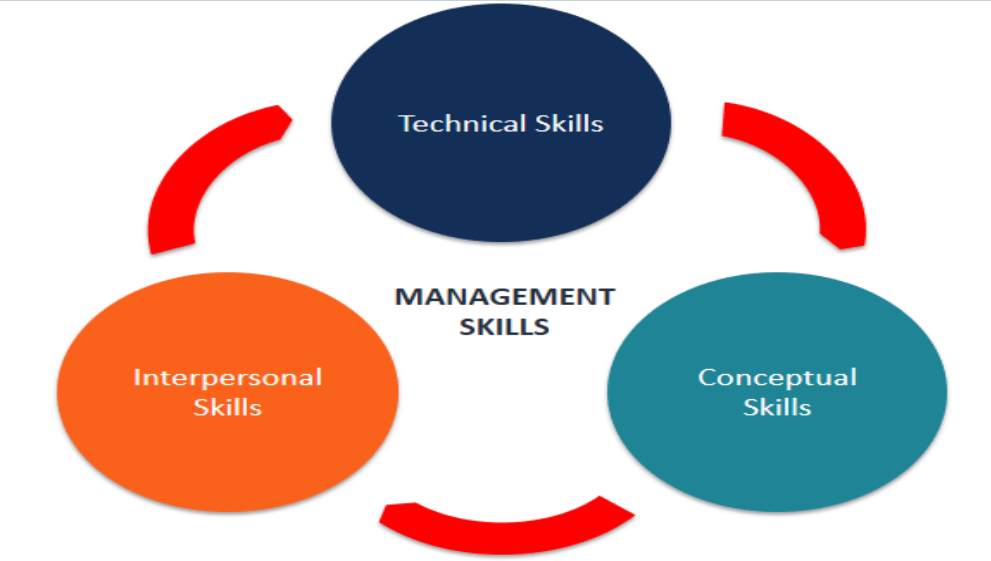
1.4 project stakeholders

- **Software Project Stakeholder** refers to, "a person, group or company that is directly or indirectly involved in the project and who may affect or get affected by the outcome of the project".



1.5 key managerial skills

- Key managerial skills for a software project include:
  - strong communication,
  - leadership,
  - effective time management,
  - problem-solving,
  - risk management,
  - collaboration,
  - adaptability,
  - critical thinking,
  - technical understanding,
  - and the ability to plan and prioritize tasks;



ensuring clear communication with stakeholders, managing project timelines, and navigating potential issues within the development team are crucial aspects of successful software project management.

Tips: Key challenges in software engineering

- The software couldn't be scaled up for complex system (scalability problem)
- Late delivery and over budget
- Poor quality (waits a lot, spend lots of money but no quality)
- Security problems
- Unreliability

Software engineering was coined to give remedy for the software crisis

- Many methods were developed:-
  - Process oriented methods (data flow diagrams)
  - Data oriented methods (E-R diagrams)
  - Object oriented methods (object oriented model)

Thank you

Questions?